

# Assignment 3: End-to-End MLOps Pipeline Report

Git Workflow, DVC Automation, Google Drive Remote & Conflict Resolution

**Student:** Muhammad Musab (Roll No: 23L-0806)  
**Repo:** github.com/codesbymusab/fashion-ann-pipeline  
**Remote:** gdrive://fashion-ann-dvc-store (1FdsWzNpKhXq...)

**Executive Summary:** Successfully implemented full end-to-end versioning across Git and DVC. Built a modular Fashion-MNIST ANN pipeline, integrated a Google Drive remote with OAuth test credentials, tracked experiments across **v1** and **v2**, and demonstrated reproducible merge conflict resolution in code and DVC locks.

Part A: Advanced Git Workflows & Demonstrations

Tasks A1 – A5

## A1 & A2: Initialization & Branching

Initialized repository with **main** branch, created **README.md** with initial typo (**Pipelnie**), configured **.gitignore** (ignoring **venv/**, **\_\_pycache\_\_**, **\*.h5**), and created **dev** branch for iterative development.

## A5: Git Stash & Branch Switching

Modified **src/preprocess.py** with a debug print without committing. Git blocked branch switching to **main** to prevent overwriting uncommitted changes. Stashed work with **git stash push**, cleanly switched to **main**, and subsequently restored on **dev** using **git stash pop**.

```
Git Bash: Git Stash & Restore Workflow (A5)

$ git status
On branch dev
Changes not staged for commit:
  Modified:   src/preprocess.py

$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
  src/preprocess.py
Please commit your changes or stash them before you switch branches.
Aborting

$ git stash push -m "wip: debug print in preprocess"
Saved working directory and index state On dev: wip: debug print in preprocess

$ git stash list
stash@0: On dev: wip: debug print in preprocess

$ git checkout main
Switched to branch 'main'

$ git checkout dev && git stash pop
Switched to branch 'dev'
On branch dev
Changes not staged for commit: modified: src/preprocess.py
Dropped refs/stash@0 (f357f85b8af65792504f32d5dec5be7eaa571c)
```

Figure 1: Git stash blocking, push, list and pop lifecycle (A5).

## A4: Git Diff Variants & Two-Dot vs Three-Dot Analysis

```
Git Bash: Diff Variants (A4)

$ git diff
diff --git a/src/preprocess.py b/src/preprocess.py
--- a/src/preprocess.py
+++ b/src/preprocess.py
@@ -46,6 +46,7 @@ def main():
     print(f"Saved processed data to {OUT_DIR}...")
+    print(f"[debug] val fraction = (len(x_val) / len(x_train):.2%)")

$ git add src/preprocess.py && git diff --staged
diff --git a/src/preprocess.py b/src/preprocess.py
--- a/src/preprocess.py
+++ b/src/preprocess.py
@@ -46,6 +46,7 @@ def main():
     print(f"Saved processed data to {OUT_DIR}...")
+    print(f"[debug] val fraction = (len(x_val) / len(x_train):.2%)")

$ git diff main...dev --stat
README.md      | 2 +-
params.yaml    | 10 ++++++++
requirements.txt | 5 +++++
src/evaluate.py | 40 +++++++++++++++++++++++++++++++++++++
src/prepare.py  | 25 +++++++++++++++++++++++++++++++++++++
src/preprocess.py | 53 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
src/train.py    | 57 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
7 files changed, 191 insertions(+), 1 deletion(-)
```

Figure 3: Unstaged, staged, and branch comparison diffs (A4).

## A3: Log Variants Analysis

```
Git Bash: Log Variants & Topology (A3)

$ git log --oneline --graph --all
* 73764d1 chore: add debug print of validation fraction in preprocess
* 202b4d3 docs: mention Git + DVC in README title
* 822a22f chore: add requirements.txt
* 2e0f49f feat(B4): add evaluate.py writing metrics.json and confusion matrix
* 2639fec feat(B3): add train.py with params-driven ANN training
* 7668120 feat(B2): add preprocess.py with normalization and validation split
* 76bdf3d feat(B1): add prepare.py to download and save raw Fashion-MNIST
* 95821bf feat: add params.yaml with preprocess and train hyperparameters
| * 32b8d9b Merge hotfix/readme-tyo into main
|/|
| * 245ca7a fix: correct README title typo
|/
* 23c3a81 chore: initial commit with README and .gitignore

$ git log main..dev --oneline
73764d1 chore: add debug print of validation fraction in preprocess
202b4d3 docs: mention Git + DVC in README title
822a22f chore: add requirements.txt
2e0f49f feat(B4): add evaluate.py writing metrics.json and confusion matrix
2639fec feat(B3): add train.py with params-driven ANN training
7668120 feat(B2): add preprocess.py with normalization and validation split
76bdf3d feat(B1): add prepare.py to download and save raw Fashion-MNIST
95821bf feat: add params.yaml with preprocess and train hyperparameters
```

Figure 2: Commit topology graph and commit range logs (A3).

| Command                                      | Unique Value Demonstrated                                                         |
|----------------------------------------------|-----------------------------------------------------------------------------------|
| <code>git log --oneline --graph --all</code> | Visualizes branch divergence, merges, and relative topology.                      |
| <code>git log --stat -3</code>               | Displays touched files and insertion/deletion counts per commit.                  |
| <code>git log -p -1</code>                   | Shows complete line-by-line patch diff of the latest commit.                      |
| <code>git log main..dev</code>               | Filters only unmerged commits present on <b>dev</b> but absent from <b>main</b> . |

### Two-Dot ( **main..dev** ) vs Three-Dot ( **main...dev** ) Difference:

- Two-dot diff ( `git diff main..dev` ):** Directly compares the tip commits of both branches. It incorporates all differences between them, meaning hotfixes or commits made exclusively on **main** will appear as negative/reversed changes.
- Three-dot diff ( `git diff main...dev` ):** Compares **dev** directly against the **merge-base** (common ancestor commit where **dev** was branched). It isolates exclusively the work accomplished on **dev**, filtering out unrelated changes from **main**.

# Assignment 3: End-to-End MLOps Pipeline Report

Git Operations (A6–A8) & Fashion-MNIST Pipeline (Part B)

Student: Muhammad Musab (Roll No: 23L-0806)

Page: 2 of 4

## Part A Continued: Rebase, Reset & Organization

## Tasks A6 – A8

### A6: Hotfix Branch, Merge & Rebase Conflict

Branched `hotfix/readme-typo` from `main`, corrected the title, and merged with `--no-ff` to preserve merge history. Rebasing `dev` onto `main` encountered a content conflict on `README.md` line 1. Resolved by retaining both corrected spelling and feature description, creating a clean linear history.

```
Git Bash: Rebase Conflict & Resolution (A6)

$ git checkout dev
Switched to branch 'dev'
$ git rebase main
Rebasing (1/8)...
Rebasing (7/8)... Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
error: could not apply 202b4d3... docs: mention Git + DVC in README title

--- Inside README.md conflict markers ---
<<<<< HEAD
# Fashion-MNIST ANN Pipeline
=====
# Fashion-MNIST ANN Pipeline (Git + DVC)
>>>>> 202b4d3 (docs: mention Git + DVC in README title)

$ cat << 'EOF' > README.md
# Fashion-MNIST ANN Pipeline (Git + DVC)
End-to-end ML versioning with Git, DVC and Google Drive.
EOF
$ git add README.md && git rebase --continue
Successfully rebased and updated refs/heads/dev.
* b3494e (HEAD -> dev) chore: add debug print of validation fraction...
* e758fe0 docs: mention Git + DVC in README title
* 32b8d9b (main) Merge hotfix/readme-typo into main
```

Figure 4: Rebase conflict stoppage, marker inspection, and linear continuation (A6).

### A8: Atomic File Moves & Deletion

Added helper `check_data.py` and notes `scratch_notes.txt`. Executed `git mv check_data.py src/check_data.py` and `git rm scratch_notes.txt`. `git show --stat -M HEAD` verified Git detected a 100% atomic rename rather than a delete/add pair.

```
Git Bash: File Reorganization with git mv and rm (A8)

$ git add check_data.py scratch_notes.txt
$ git commit -m 'chore: add loose helper script and scratch notes'
[dev 3212e6e] chore: add loose helper script and scratch notes

$ git mv check_data.py src/check_data.py
$ git rm scratch_notes.txt
rm 'scratch_notes.txt'

$ git status
Changes to be committed:
  Deleted:    scratch_notes.txt
  Renamed:    check_data.py -> src/check_data.py

$ git commit -m 'refactor: move check_data.py into src/ and remove obsolete notes'
$ git show --stat -M HEAD
commit adb3e6e42ffdb25e5849e309e759dae229f09fb
Author: Musab <codesbymusab@gmail.com>
  refactor: move check_data.py into src/ and remove obsolete notes
  scratch_notes.txt | 1 -
  check_data.py -> src/check_data.py | 0
  2 files changed, 1 deletion(-)
  rename check_data.py -> src/check_data.py (100%)
```

Figure 6: Git atomic rename tracking and file deletion (A8).

### A7: Soft Reset vs Hard Reset Scenarios

```
Git Bash: Soft vs Hard Reset Scenarios (A7)

$ git checkout -b scratch
$ echo 'experiment 1' > scratch1.txt && git add scratch1.txt && git commit -m 'scratch: experiment 1'
[scratch 9cf811e] scratch: experiment 1

$ git reset --soft HEAD~1
$ git status
On branch scratch
Changes to be committed: (use 'git restore --staged <file>...' to unstage)
  Bnew file:   scratch1.txt
  $ ls scratch1.txt
scratch1.txt

$ git commit -m 'scratch: experiment 1 (recommitted)'
$ echo 'experiment 2' > scratch2.txt && git add scratch2.txt && git commit -m 'scratch: experiment 2'
$ git reset --hard HEAD~1
HEAD is now at b415238 scratch: experiment 1 (recommitted)
$ git status
nothing to commit, working tree clean
$ ls scratch2.txt
ls: cannot access 'scratch2.txt': No such file or directory
```

Figure 5: Soft reset (staged on disk) vs hard reset (purged working tree) (A7).

**Reset Mechanics:** `--soft` shifts only the `HEAD` pointer back one commit; changes remain indexed (staged) and preserved on disk. `--hard` shifts `HEAD` and resets both index and working tree, permanently discarding changes from disk.

## Part B: Fashion-MNIST ANN Pipeline

## Tasks B1 – B4

Built a 4-stage modular architecture reading from `params.yaml1`:

- **B1** `src/prepare.py`: Loads raw Fashion-MNIST, saves deterministic `.npy` files to `data/raw/`.
- **B2** `src/preprocess.py`: Scales pixels to `[0, 1]`, carves 10% stratified validation split, saves to `data/processed/`.
- **B3** `src/train.py`: Compiles sequential ANN ( `Flatten` → `Dense(ReLU)` → `Dropout(0.2)` → `Dense(10, Softmax)` ). Logs history to `history.csv`, saves `models/model.h5`.
- **B4** `src/evaluate.py`: Evaluates test set, writes `metrics.json` and saves `confusion_matrix.png`.

**Baseline Training Performance:** Validation accuracy converged to **89.02%** with validation loss of **0.3064** over 10 epochs.

# Assignment 3: End-to-End MLOps Pipeline Report

DVC Cloud Remote (Part C) & Pipeline Automation (Part D)

Student: Muhammad Musab (Roll No: 23L-0806)

Page: 3 of 4

## Part C: DVC Remote Storage with Google Drive

Tasks C1 – C5

Configured Google Drive as remote storage

(`gdrive://1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS`). Implemented custom Google Cloud OAuth Desktop Application credentials to avoid public shared-client access blocks.

**Credential Security Verification:** The client secret was configured with `--local`, isolating it inside `.dvc/config.local`. Running `git check-ignore -v .dvc/config.local` confirms it is ignored by Git. `git ls-files | grep -iE 'cred|token|secret'` confirms 0 credentials leaked.

```
Git Bash: DVC Init, Google Drive Remote & Push (Part C)

$ dvc init
Initialized DVC repository.
$ dvc remote add -d gdrive_storage gdrive://1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS
Setting 'gdrive_storage' as a default remote.

$ cat .dvc/config
[core]
  remote = gdrive_storage
[remote "gdrive_storage"]
  url = gdrive://1FdsWzNpKhXq4zAcrmCMqHFjtrnoguXLS
  gdrive_client_id = 441442683644-m8aohr1itg60lbaknjhue0dlubn9mqee.apps.googleusercontent.com

$ git check-ignore -v .dvc/config.local
.dvc/.gitignore!:/config.local!dvc/config.local

$ git ls-files | grep -iE 'cred|token|secret'
(No results found - credentials strictly uncommitted)

$ dvc push
14 files pushed
$ dvc status -c
Cache and remote 'gdrive_storage' are in sync.
```

Figure 7: DVC remote configuration, credential check, and 14 files pushed to Drive (Part C).

## Part D: DVC Pipeline & Experiments

Tasks D1 – D5

Handed artifact tracking from `.dvc` files to `dvc.yaml`. Authoring explicit dependencies and parameter locks ensures deterministic stage execution.

```
Git Bash: DVC Pipeline Reproduction & Diff (Part D)

$ dvc repro (Version 1 baseline)
Running stage 'prepare': > python src/prepare.py
Running stage 'preprocess': > python src/preprocess.py
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3605 test_accuracy=0.8666
Updating lock file 'dvc.lock'
$ git tag v1 && dvc push

$ dvc params diff
Path      Param      HEAD      workspace
params.yaml train.dense_units 128       256

$ dvc repro (Version 2 with 256 dense units)
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3447 test_accuracy=0.8774
Updating lock file 'dvc.lock'

$ dvc metrics diff v1 v2 --md
| Path      | Metric      | v1      | v2      | Change  |
|-----|-----|-----|-----|-----|
| metrics.json | test_accuracy | 0.8666  | 0.8774  | 0.0108  |
| metrics.json | test_loss    | 0.36053 | 0.34467 | -0.01586 |
```

Figure 8: Baseline run (v1), parameter alteration, stage skipping, and v2 diff (Part D).

## D4: Stage Skipping Analysis & Metric Comparison (v1 vs v2)

| Metric / Param    | v1 (128 units)  | v2 (256 units)  | Observed Change     |
|-------------------|-----------------|-----------------|---------------------|
| train.dense_units | 128             | 256             | +128 units          |
| test_accuracy     | 0.8666 (86.66%) | 0.8774 (87.74%) | +0.0108 (+1.08%)    |
| test_loss         | 0.36053         | 0.34467         | -0.01586 (Improved) |

### Why DVC Skipped Stages during v2 ( `dvc repro` ):

DVC tracks content hashes of code, inputs, and parameters in `dvc.lock`. When `train.dense_units` changed: 1. `prepare` & `preprocess` depend only on raw data and preprocessing params (unchanged) → **Skipped**. 2. `train` depends on `train.dense_units` → **Re-executed**, producing new `model.h5`. 3. `evaluate` depends on `model.h5` → **Re-executed** automatically.

# Assignment 3: End-to-End MLOps Pipeline Report

Collaboration Simulation, Conflict Resolution & Reproducibility (Part E)

Student: Muhammad Musab (Roll No: 23L-0806)

Page: 4 of 4

## Part E: Simulated Collaboration & Dual Conflict Resolution

## Tasks E1 – E5

### E1 & E2: Parallel Feature Branches

Merged `dev` to `main`. Branch `teammate-sim` implemented Fashion-MNIST standardization:  $(x - 0.2860) / 0.3530$ . Branch `main` concurrently implemented centering:  $(x / 255.0) - 0.5$ . Both regenerated data via `dvc repro preprocess` and pushed to Google Drive.

### E3: Dual Conflict (Code & DVC Lock)

Merging `teammate-sim` triggered two conflicts: a code conflict in `src/preprocess.py` and an md5 collision in `dvc.lock` for `data/processed`.

```
Git Bash: Merge Conflict in Code & DVC Lock (Part E)

$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.

$ git diff src/preprocess.py
++<<<<<< HEAD
+   """Main approach: scale to [0, 1], then center to [-0.5, 0.5]."""
+   return x.astype("float32") / 255.0 - 0.5
+
+
+
+   """Teammate approach: scale to [0, 1], then standardize with Fashion-MNIST mean/std."""
+   return (x - 0.2860) / 0.3530
++>>>>>> teammate-sim

$ git diff dvc.lock
++<<<<<< HEAD
+   md5: a5957418b4fec2d191a841c36800955e.dir
+
+
+   md5: 19e07b0684b365c7732232348a6d2257.dir
++>>>>>> teammate-sim

--- Resolution Steps ---
$ # Reconciled src/preprocess.py with standardization + scaling
$ git add src/preprocess.py
$ git checkout --ours dvc.lock && git add dvc.lock && dvc checkout
$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess': > python src/preprocess.py
Running stage 'train': > python src/train.py
Running stage 'evaluate': > python src/evaluate.py
test_loss=0.3793 test_accuracy=0.8715
$ git commit -m 'Merge teammate-sim into main: reconcile normalization, regenerate data'
$ dvc repro
Data and pipelines are up to date.
```

Figure 9: Conflict markers in code and dvc.lock, reconciliation, and data regeneration (Part E).

### E4 & E5: Authoritative Resolution & Reproducibility

1. **Code:** Unified both approaches into a shared scale-then-standardize function:

```
FASHION_MEAN, FASHION_STD = 0.2860, 0.3530
def normalize(x):
    x = x.astype("float32") / 255.0
    return (x - FASHION_MEAN) / FASHION_STD
```

2. **Data Lock:** Because the merged code generates a *new 3rd version* of data, cleared conflict markers with `git checkout --ours dvc.lock`, synced workspace with `dvc checkout`, and regenerated with `dvc repro` (final accuracy: **87.15%**).

3. **Reproducibility Proof:** Running `dvc repro` immediately after merge confirmed all stages print "*didn't change, skipping*".

```
Git Bash: Final Topology Graph (Part E)

$ git log --oneline --graph --all
* c53d19f (HEAD -> main, origin/main) Merge teammate-sim into main: reconcile normalization...
| \
| * b3122eb (origin/teammate-sim, teammate-sim) feat(E1): standardize pixels with mean/std
* | 6a4682d feat(E2): center pixels to [-0.5, 0.5] (main)
| \
| * 397a876 (tag: v2, origin/dev, dev) exp(D5): increase dense_units 128 -> 256 (v2)
* 0940db5 (tag: v1) feat(D3): add dvc.yaml pipeline and first reproduced run (v1)
* a0b02ca chore(D): hand artifact tracking over to dvc.yaml stages
* 8c6d53 chore: use custom Google OAuth client for DVC remote
* 058083f feat(C5): track raw data, processed data and model with DVC
* a6437f1 chore(C3): configure Google Drive as default DVC remote
* 8975788 chore(C2): initialize DVC
* adb36e refactor: move check_data.py into src/ and remove obsolete notes
* 321266 chore: add loose helper script and scratch notes
* b34048c chore: add debug print of validation fraction in preprocess
* a758fe0 docs: mention Git + DVC in README title
* 032a923 chore: add requirements.txt
* 3cd13e feat(B4): add evaluate.py writing metrics.json and confusion matrix
* 6fec6ad feat(B3): add train.py with params-driven ANN training
* ba78f28 feat(B2): add preprocess.py with normalization and validation split
* aac54e9 feat(B1): add prepare.py to download and save raw Fashion-MNIST
* ed5649d feat: add params.yaml with preprocess and train hyperparameters
* 32b8d9b Merge hotfix/readme-typo into main
| \
| * 245ca7a fix: correct README title typo
| \
* 23c3aa1 chore: initial commit with README and .gitignore
```

Figure 10: Complete unsquashed Git topology with tags, rebase, and merge commit.

## Final Deliverables & Verification Checklist

## Summary

### GitHub Repository:

- Branches: `main`, `dev`, `teammate-sim`
- Tags: `v1` (baseline), `v2` (256 units)
- Unsquashed, linear + merge history

### DVC Remote Storage:

- Remote: Google Drive (ID: `1FDsWzNpKh...`)
- Status: *Cache & remote in sync*
- Zero credentials committed to Git

### Pipeline Artifacts:

- `dvc.yaml` & `dvc.lock` committed
- `metrics.json` & `confusion_matrix.png`
- 100% reproducible execution